JAS OS Technical Report

Field	Details
Project	JAS OS v1.5.4
Student	Jubayed Ahmed Sahel
Student ID	2221173642
Course	CSE 323, Section 3
Repository	JAS OS on GitHub https://github.com/Jubayed-Sahel/JAS-OS

Abstract

JAS OS is a bootable, freestanding 32-bit x86 operating-system project. It combines a custom BIOS/El Torito boot path, a protected-mode kernel, a graphical desktop, a terminal, and an OS Lab interface. The project makes core operating- system concepts observable through live state rather than static interface labels, including task scheduling, synchronization, memory allocation, paging, deadlock avoidance, file management, storage algorithms, and I/O models.

At a glance: JAS OS turns operating-system theory into an interactive x86 system. Boot, kernel services, algorithms, and interface controls are connected so that each concept can be demonstrated and measured directly.

Introduction

Operating-system concepts are easiest to understand when their results can be observed while the system is running. JAS OS was developed to provide that practical view within a self-contained x86 environment. Instead of presenting only screenshots or descriptive labels, the project connects each major concept to an executable kernel module and exposes its changing state through the terminal and graphical OS Lab interface.

The project has three primary objectives. First, it establishes a complete boot path from a BIOS-compatible ISO to a 32-bit protected-mode kernel. Second, it implements representative operating-system mechanisms, including scheduling, synchronization, memory management, paging, file management, storage, and I/O. Third, it provides a clear demonstration path through which a user can trigger those mechanisms and inspect their results. These objectives make the system useful both as a working course project and as a repeatable platform for explaining operating-system behavior.

The report describes the system's architecture, implemented features, engineering challenges, and verification results. It also identifies the boundaries of the implementation so that the educational models are not mistaken for production operating-system facilities.

1. Executive Summary

JAS OS is a bootable, freestanding 32-bit x86 operating system developed as a CSE 323 course project. It boots from an ISO in Oracle VirtualBox and provides an interactive desktop, terminal, applications, and observable kernel subsystems. The system exposes task states, scheduling activity, synchronization values, memory statistics, deadlock-safety results, paging state, file-system metadata, storage calculations, and I/O counters as live evidence of implementation behavior.

2. System Overview and Scope

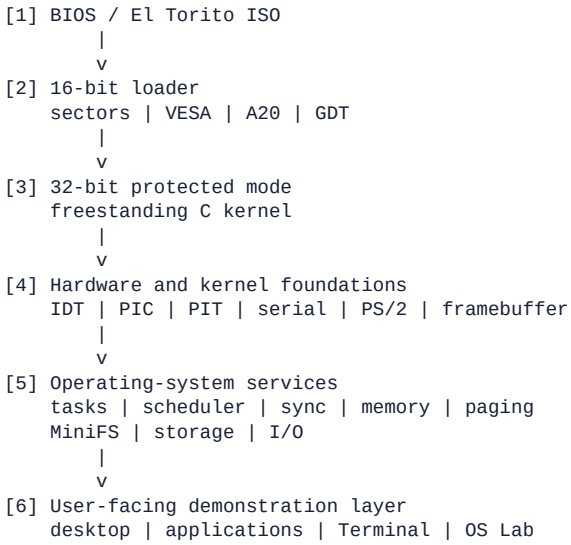
JAS OS is designed as a PC-style x86 platform with a custom boot path, a protected-mode kernel, a graphical desktop, Terminal, Files, Notes, Task Manager, Calculator, Settings, Clock, and OS Lab.

The kernel includes scheduling, synchronization, deadlock avoidance, memory allocation, paging, file-system, storage, and I/O modules. The implementation is deliberately educational: tasks are cooperative, applications execute in kernel space, MiniFS is RAM-backed, and paging, disk, RAID, and I/O are software models. The project does not claim user-mode isolation, networking, production hardware drivers, or permanent disk-backed persistence.

3. Architecture and Implemented Features

The boot sequence establishes the execution environment before the kernel initializes its device and service layers:

How to read the architecture: The numbered flow moves from left-to-right boot preparation into kernel services and finally into the user-facing demonstration layer. The table that follows explains what each layer does and what evidence a reader can observe.



Each layer has a distinct responsibility and a visible outcome:

Layer	Responsibility	Visible outcome
Boot	Load the kernel and establish protected mode	Boot trace and working ISO
Hardware foundations	Initialize interrupts, timer, input, serial, and graphics	Responsive desktop and diagnostic output
Kernel services	Manage tasks, resources, files, storage, and I/O	Live counters, states, and algorithm results
Demonstration layer	Expose services through commands and controls	Repeatable terminal and OS Lab workflows

The principal feature areas are summarized below.

Feature	Evidence in JAS OS
OS organization	Live subsystem dashboard
Services and boot	BIOS-to-protected-mode trace and syscall aliases
Processes and IPC	Task states and bounded mailbox send/receive
Threads	Shared-counter workers
CPU scheduling	RR, Priority, FCFS, SJF, and timeline
Synchronization	Semaphores and readers-writers
Deadlock avoidance	Banker's safety algorithm and safe sequence
Main memory	First-fit heap, splitting, and coalescing

Feature	Evidence in JAS OS
Virtual memory	Paging, faults, frames, and FIFO replacement
File system	MiniFS directories, files, and allocation comparison
Mass storage	Disk-scheduling algorithms and RAID model
I/O systems	Polling, interrupts, DMA, and live counters

4. Demonstration Interface

The terminal Feature Guide and Modules screen connect each subsystem to a direct command. A scheduling-policy change is reflected in the CPU timeline; the readers-writers demonstration updates semaphore-protected state; Banker's algorithm produces a safe sequence; and paging updates frame ownership and fault counts. OS Lab exposes the same kernel operations through graphical controls. The global `stop` command terminates non-shell activity while keeping the terminal available for further interaction.

5. Engineering Challenges and Solutions

The following challenges are documented using the Situation, Task, Action, and Result (STAR) structure.

Challenge 1 — Building a bootable x86 ISO

Situation: Before starting the x86 version, I was developing an ESP32-S3 mini-kernel for CSE 323. That earlier work ran on a microcontroller platform, but this project required a PC operating system that could boot independently from an ISO in Oracle VirtualBox.

Task: Transfer the relevant operating-system concepts into a freestanding x86 environment without a host operating system or standard library, while also providing keyboard, mouse, and graphical output.

Action: The implementation introduced a BIOS/El Torito loader, VESA framebuffer setup, A20 handling, a GDT, and a protected-mode transition. It then added IDT/PIC/PIT support, serial output, PS/2 input, graphics, and freestanding runtime modules.

Result: JAS OS boots from its own ISO into an interactive 1024x768 desktop, and the cross-compiler build completes without warnings or errors.

Challenge 2 — Producing live evidence instead of static labels

Situation: Static interface labels would not show whether the algorithms were connected to changing system state.

Task: Connect every major subsystem to stateful code and observable results.

Action: Scheduling, tasks, synchronization, memory, paging, Banker's algorithm, files, storage, and I/O were separated into focused kernel modules. The interface exposes task states, timelines, semaphore values, heap statistics, safe sequences, page faults, disk-head movement, and I/O counters.

Result: Each feature has a terminal command and OS Lab button backed by live kernel state, making state changes and algorithm results directly observable.

Challenge 3 — Organizing demonstrations during assessment

Situation: The command set became difficult to remember, and background output could hide the result being discussed.

Task: Make implementation evidence easy to find without replacing the underlying algorithms with static interface text.

Action: The Feature Guide, Modules screen, readable terminal layout, scrolling controls, and OS Lab shortcuts were added. Each path leads to a direct command for a specific kernel subsystem.

Result: The interface now provides a consistent route from a feature name to its source-backed live result.

Challenge 4 — Stopping every background task safely

Situation: Producer-consumer, thread, readers-writers, and counter jobs can all generate asynchronous terminal output. The old stop action controlled only one demonstration.

Task: Stop all background activity without terminating the shell.

Action: The global stop handler calls each laboratory's normal shutdown, scans the task table, terminates remaining non-shell tasks, and restores scheduler state. Controls for individual laboratories remain available.

Result: `stop`, `stop all`, `lab stop`, and the red STOP button end terminal background activity while the shell remains ready for the next command.

Challenge 5 — Debugging without normal operating-system tools

Situation: A freestanding kernel cannot depend on a host terminal, standard library, desktop debugger, or operating-system error dialog while it is running. A small mistake in initialization can therefore prevent the system from reaching the graphical interface at all.

Task: Locate failures in boot, interrupt handling, input, and kernel logic with limited runtime visibility.

Action: Serial output and the framebuffer were used as complementary diagnostic channels. Initialization stages were kept explicit, and the terminal's live status commands were used to inspect scheduler, memory, paging, storage, and I/O state after the system booted.

Result: Failures could be narrowed to a boot stage or subsystem instead of being treated as an unexplained blank screen. This also made the same diagnostic information useful during the final demonstration.

Challenge 6 — Connecting kernel state to a usable interface

Situation: Implementing an algorithm was only part of the assignment. The result also had to be understandable to someone viewing the desktop and terminal during a short assessment.

Task: Present changing kernel state without duplicating the implementation or replacing it with hard-coded demonstration output.

Action: Commands were designed around the state already maintained by each kernel module. The terminal and OS Lab then invoked those commands, while status views displayed values such as task states, semaphore counts, heap statistics, page faults, and I/O counters.

Result: The interface became an inspection layer over the kernel rather than a separate collection of simulated screens. A feature could be demonstrated, examined, and explained using the same underlying state.

Challenge 7 — Managing breadth within a student project

Situation: The project covered many syllabus areas, but each additional feature increased the chance of regressions in shared code such as task management, terminal input, and scheduling.

Task: Deliver broad coverage while keeping the system bootable and the demonstration route predictable.

Action: Features were organized into separate kernel modules with explicit commands and shutdown paths. The demonstration route was kept short, and stateful commands were tested with both successful and invalid inputs before being included in the final workflow.

Result: The project achieved broad operating-system coverage without making the assessment depend on an unstructured sequence of manual steps. The modular boundaries also made individual problems easier to isolate and correct.

6. Verification and Results

• The freestanding kernel builds without warnings or errors using the included

`i686-elf` toolchain.

• The generated ISO boots in a 32-bit BIOS VirtualBox VM at 1024x768.

• The terminal regression suite passed 208 command and power checks.

• The terminal and OS Lab expose each major subsystem through live state.

• The global stop action leaves the terminal shell responsive.

• The narrated demonstration uses genuine JAS OS captures and remains within

the required two-to-five-minute range.

Reproduction procedure

• Build the ISO with `powershell -ExecutionPolicy Bypass -File .\build.ps1`.

• Start a BIOS-mode VirtualBox VM configured as **Other/Unknown (32-bit)**.

• Attach the generated `jas-os.iso` as the optical disk and boot the VM.

• Open Terminal and run `guide`, `status`, or any feature command listed in

the repository README.

This procedure supports transparent classroom evaluation: each command routes to the corresponding kernel module, and resulting state changes can be observed in the terminal or OS Lab interface.

7. GitHub Links

Resource	Direct link
Repository	JAS OS on GitHub https://github.com/Jubayed-Sahel/JAS-OS
Source code	Browse the kernel source https://github.com/Jubayed-Sahel/JAS-OS/tree/main/kernel
Project README	Read the project overview https://github.com/Jubayed-Sahel/JAS-OS/blob/main/README.md
Technical report	Markdown report https://github.com/Jubayed-Sahel/JAS-OS/blob/main/docs/TECHNICAL_REPORT.md · PDF report https://github.com/Jubayed-Sahel/JAS-OS/blob/main/docs/JAS-OS-Technical-Report.pdf
Demonstration	Watch the demonstration video https://github.com/Jubayed-Sahel/JAS-OS/blob/main/Demonstration%20video.mp4

8. Project Deliverables

• GitHub repository:

• Bootable ISO: `jas-os.iso`

• Narrated demonstration: Watch the demonstration video
https://github.com/Jubayed-Sahel/JAS-OS/blob/main/Demonstration%20video.mp4

9. Limitations and Scope Boundary

JAS OS is an educational course-project kernel, not a production operating system. Tasks are cooperative, applications execute in kernel space, and MiniFS is RAM-backed. Paging, disk, RAID, and I/O are transparent software models. Accordingly, the project does not claim ring-3 isolation, production device drivers, networking, persistent storage, or production security.

10. Conclusion

JAS OS demonstrates that a compact freestanding x86 kernel can present a broad set of operating-system concepts through one coherent, interactive platform. The project brings together bootstrapping, protected-mode execution, hardware initialization, scheduling, synchronization, memory management, paging, file management, storage, and I/O. Each area is connected to live commands and graphical controls, allowing system behavior to be examined as it changes.

The most significant outcome is the connection between implementation and evidence. A task table exposes scheduler state, demonstrations reveal semaphore and shared-resource behavior, Banker's algorithm reports safety decisions, and memory, paging, storage, and I/O modules publish measurable results. The global stop mechanism further demonstrates that the interactive environment can manage concurrent demonstrations without sacrificing shell responsiveness.

The resulting system meets its educational purpose while maintaining a clear technical boundary: it is a cooperative, kernel-space course project rather than a production operating system. Within that scope, the bootable ISO, interactive tools, documented commands, and verification results provide a credible and repeatable demonstration of the underlying operating-system principles.